1. Introduction

Modern data-intensive applications rely on distributed systems to process large datasets efficiently. Instead of storing and querying all on a single machine, data is partitioned across multiple nodes to allow parallel processing and improved scalability. However, distributed query execution introduces challenges such as coordination overhead, memory management, load balancing, and maintaining efficient communication between nodes.

This project extends a single-node query system into a multi-node distributed architecture using gRPC for inter-process communication. A network of processes (nodes A-I) are connected through a predefined overlay, in which each node owns a data subset. Queries are submitted through a single entry point and executed across the overlay, with results aggregated and returned to the client.

The objective of this project is to design, implement, and evaluate a distributed query system that executes queries across multiple nodes without data replication, uses a structured overlay network for request forwarding and aggregation, supports segmented responses through chunk-based retrieval, and balances performance, memory usage, and fairness across nodes. A baseline system and an optimized system are evaluated to then analyze the trade-offs between eager and on-demand execution.

2. System Architecture Overview

The system is a distributed ingestion and query processing platform designed for NY taxi trip data. As shown in **Figure 1**, it consists of four main components: dataset partitioning, ingestion control, distributed query coordination, and local query execution.

In the first stage, raw CSV data is parsed and divided into smaller data shards, which act as the basic units of distribution. The ingestion layer then routes these shards to worker nodes using a queue-based mechanism, enabling controlled and decoupled data distribution.

When a query is submitted, the query coordinator (Node A) receives the request and distributes it across worker nodes. Each worker executes the query locally on its assigned shard and returns partial results. These results are then aggregated by the coordinator to produce the final response, which is sent back to the client.This design enables distributed query execution across multiple nodes while keeping data processing localized to each shard.

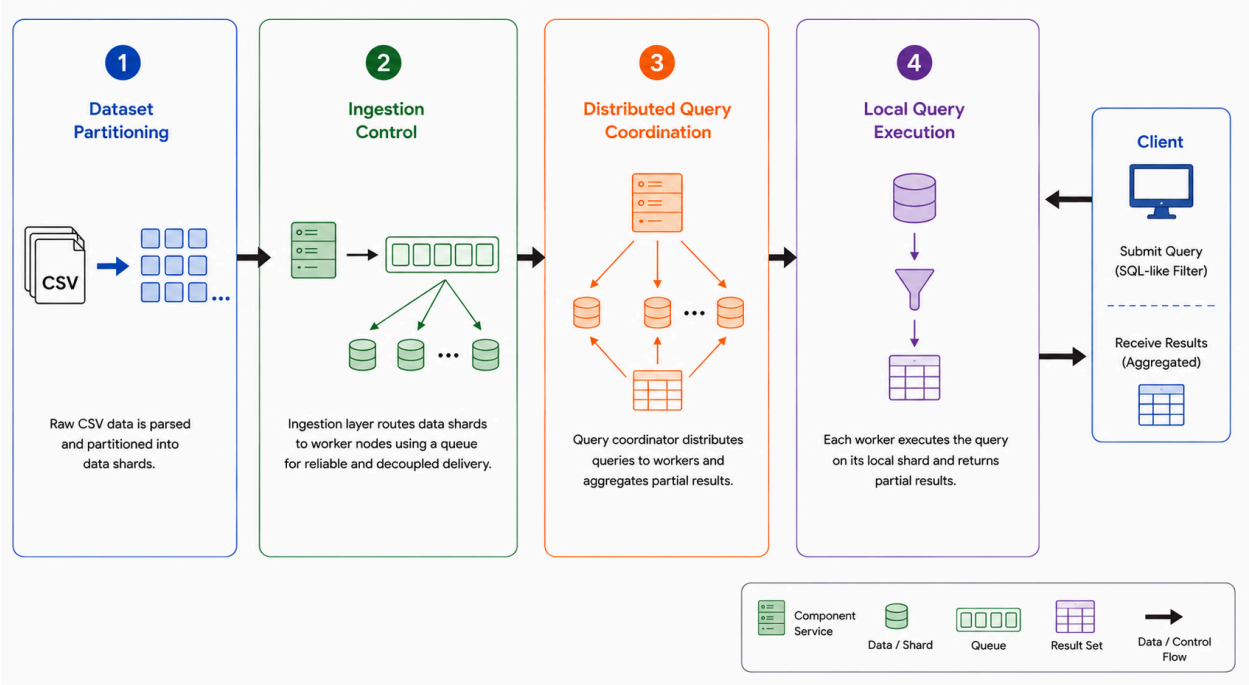


Figure 1: High-Level System Architecture

2.1 Distributed Node Organization and Tree Overlay

The distributed part of the system follows a leader–worker model. The leader receives the original query request and is responsible for coordination and result aggregation, while worker nodes store their assigned partitions and execute queries locally. This maps to three core components: QueryCoordinator, WorkerNode, and RequestState, which handle scatter–gather execution, node representation, and tracking in-flight query state, respectively.

As shown in **Figure 1**, the nodes are organized as a logical tree overlay, with the leader at the root. When a query is issued, the leader scatters the QueryRequest across the worker nodes. Each worker processes the query on its local data and returns a LocalQueryResult. These partial results are then gathered back through the overlay and merged into a final QueryResult.

This approach keeps coordination separate from local execution. The leader does not touch the data directly,and the query logic remains unchanged even as new worker nodes are added to the overlay.

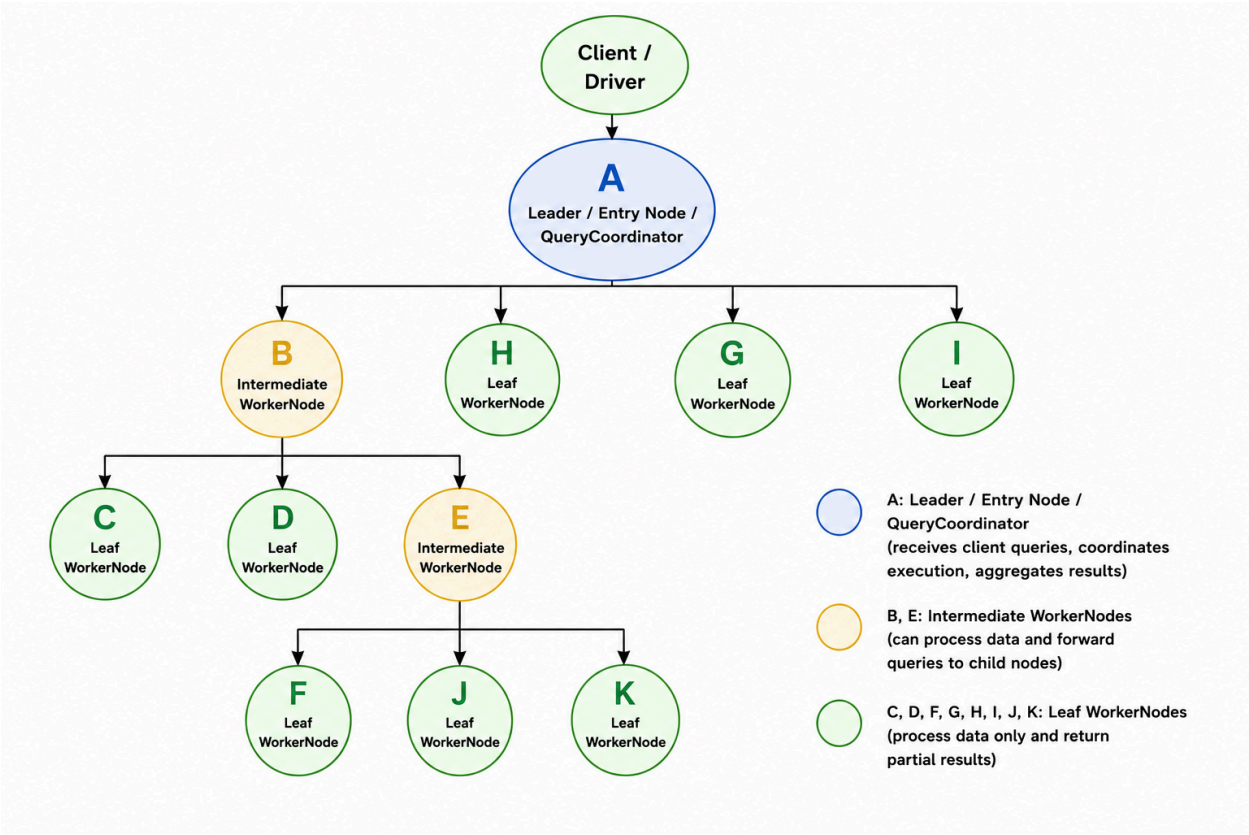


Figure 1: Distributed Overlay Topology for Query Routing and Result Aggregation

2.2 gRPC Communication and Service Interfaces

Node communication runs over gRPC. It enforces typed messages and defined service interfaces, which matters when passing queries, partition data, node metadata, and partial results between leader and workers.

The flow is straightforward: QueryCoordinator sends a QueryRequest to a worker via gRPC, the worker executes it locally and returns a LocalQueryResult, and the coordinator aggregates these into the final QueryResult. Supporting this are NodeInfo and OverlayConfig, which handle node identity and overlay configuration.

The communication layer has no overlap with query processing logic — each can be tested and modified independently.

2.3 End-to-End Data and Query Flow

- **Ingestion Flow**

CSV files are parsed into structured trip records and stored as partitions on disk. Components such as PartitionLoaderand PartitionStore handle parsing and storage, while ShardAssignment decides which

worker node should receive each partition. The ingestion layer then schedules and delivers these partitions to workers.

In the baseline version, partitions are distributed using a simple round-robin strategy. In the optimized version, assignment decisions are improved using workload-aware signals such as file size, trip count, queue pressure, and current worker load, resulting in more balanced data placement.

- **Query Flow**

A client or benchmark driver issues a QueryRequest containing filter conditions such as pickup and dropoff time, distance, tip, total amount, and payment type, along with routing metadata to ensure correct traversal across the tree overlay.

The QueryCoordinator receives the request and distributes it to worker nodes. Each worker executes the query using LocalQueryEngine, scanning its local partitions and applying filters directly to the data.

The system supports two execution modes: count queries, which return only the number of matching rows, and execute queries, which return the actual matching records. For large result sets, execution is performed in chunks using parameters such as startRow, chunkSize, and scanRowBudget, allowing workers to process bounded portions of data at a time.

Each worker returns a LocalQueryResult containing matched rows and pagination state. The coordinator gathers these partial results and merges them into the final response. This design keeps data processing local to each worker, while the coordinator handles only routing and aggregation.

3. Baseline Architecture

The baseline implementation establishes a functional distributed query system over the specified tree overlay. Its purpose is to verify end-to-end distributed query execution across all nodes (A-I), ensure each node processes only its assigned shard, and to provide a reference point for evaluating optimizations. Correctness and simplicity is prioritized over performance as it intentionally does not optimize for memory usage, fairness, or execution latency.

Data shards are distributed using a round-robin assignment strategy, resulting in a stateless assignment, each node receiving approximately the same number of files, and no consideration of file size differences. However, this can lead to load imbalance if shard sizes vary significantly. This could be seen if distributed through monthly CSV shards.

For ingestion, each node loads its assigned shard into a local storage PartitionStore. Data is parsed and stored in memory for querying by the client service. There exists no global coordination or load balancing during this stage.

The query flow starts with client submission where Node A acts as the entry point for all queries. Here, eager distributed execution is observed. Node A executes its local shard query and forwards the query recursively to all neighbors. Neighbor nodes execute their own local shard query and propagate the request towards their neighbors before all results are returned back to Node A. It should be noted that the

entire result set is computed during SubmitQuery. Furthermore, query execution is blocking and so all nodes must finish before SubmitQuery returns. Once Node A has all the results, data is aggregated into a single result set in RequestState. The client then is able to receive slices of the precomputed result via fetch, meaning no additional computation occurs.

The baseline system provides a simple implementation of distributed query execution but exhibits high submit latency due to full dataset scanning, potential load imbalance due to round-robin assignment, and high memory usage from caching full result sets. These limitations will be explored in the next section.

4. Motivation for Optimization

In the baseline design, ingestion and query execution follow a simple deterministic flow. Shards are assigned in order, and EXECUTE queries are processed in a precompute-oriented manner: each worker scans its assigned partitions, applies the filters, stores matched rows, and returns the local result set after processing completes. This made the baseline easy to validate and useful as a correctness reference.

From the baseline behavior, we identified three main improvement opportunities:

- **Uneven shard workload**

The original setup used one monthly CSV file per shard, but the files were not equal in size. For example, January was 621 MB with 6.4 million rows, while April was only 23 MB with about 238,000 rows. This caused uneven scan and I/O work across workers and motivated weighted shard assignment using file size and trip count.

- **Memory usage grows with matched rows**

Workers returned results only after scanning and storing the full local result set. This delayed partial output and increased memory usage when many rows matched a query.

- **Work is handled in large units**

Each query was handled as one continuous local task, which gave less control when multiple requests were active. This motivated chunk-based execution so large queries could progress in smaller bounded units.

These observations motivated the optimized design, which keeps the same distributed architecture but adds workload-aware shard placement, least-loaded dispatching, and bounded query execution. This helps distribute work more evenly, reduce peak memory usage, and allow active queries to make steady progress.

5. Workload-Aware Sharding and Query Execution

The optimized architecture keeps the same leader-worker and tree overlay structure, but improves shard assignment, dispatching, and query result handling. Instead of assigning work only by order, it uses shard

weight information, least-loaded dispatching, and bounded query execution. Work is still parallelized at the file-shard level, where each CSV shard is processed locally by a worker.

5.1 File-level sharding

The system uses CSV files as shard units. In the original setup, each monthly CSV file was assigned to one shard.

Since monthly files can differ in size and row count, the optimized design stores shard metadata with sourceFile, nodeId, fileSizeBytes, and tripCount before assignment.

5.2 Weighted shard assignment

The baseline assigns shards in order, which treats all shards as equal. The optimized version represents each shard using ShardAssignment, which includes sourceFile, nodeId, fileSizeBytes, and tripCount. Here, fileSizeBytes estimates I/O cost, while tripCount estimates scan cost.This allows assignment decisions to be based on workload rather than shard count alone.

5.3 Load-aware ingestion dispatching

After a shard is popped from IngestQueue,IngestDispatcher uses a least-loaded assignment instead of a simple round-robin. It tracks cumulative worker load using workerAssignedBytes_ and workerAssignedTrips_. When assigning a shard, the dispatcher selects the worker with the lowest assigned byte load, using trip count as a tie-breaker. This reduces the chance that multiple large shards are placed on the same worker.

Queue control and adaptive chunking

IngestQueue acts as a FIFO buffer for ShardAssignment jobs. It stores discovered shards and lets the dispatcher pop one shard at a time using popNext(), separating shard discovery from worker assignment. During query execution, chunkSizeand scan limits break large results into smaller bounded chunks.

5.5 Optimized query routing and segmented responses

LocalQueryEngine uses startRow, chunkSize, and scanRowBudget from QueryRequest to process results in segments. Each worker returns a bounded chunk along with nextStartRow and hasMore, allowing the QueryCoordinator to request more chunks only when needed. This reduces peak memory usage and supports incremental result delivery.

6.1 Experimental Setup - Baseline

The baseline system was evaluated using a 9-node distributed overlay consisting of nodes A through I. Each node was assigned one CSV shard, with no replication or data sharing between nodes. The client connected exclusively to Node A, which acted as the entry point for all queries and coordinated execution across the overlay.

Shards were loaded into memory at its assigned node using PartitionStore. Shard assignment followed a round-robin strategy, where each node received exactly one shard. No additional metadata such as file size or row count was used during assignment.

For query evaluation, the client submitted requests to Node A using SubmitQuery, followed by repeated FetchChunk calls to retrieve results. The system used a chunk-based response mechanism, where each fetch returned up to max_row results. Pagination was handled internally using a cursor, allowing the client to retrieve results incrementally.

To evaluate chunking behavior, experiments were conducted using different values of max_rows. For each query configuration, multiple runs were executed to ensure consistency, and results were averaged across runs.

Queries were evaluated across multiple filter conditions, including no filter, single-attribute filters, and combined filters, to ensure coverage of different query patterns.

6.2 Experimental Setup - Optimized

The optimized version was tested with 9 nodes, using shards A–I. Each node had one local CSV shard, and the client connected only to Node A. Node A worked as the entry point and collected results from the other nodes through the overlay.

Each shard was stored as a ShardAssignment with its file path, assigned node, file size, and trip count. These values were used to estimate the workload of each shard instead of treating all shards as equal.

During ingestion, shard jobs were added to IngestQueue. The dispatcher removed one shard at a time and assigned it to the least-loaded worker based on assigned bytes and trip count.

For queries, the client submitted requests to Node A and fetched results using repeated FetchChunk calls. The query used startRow, chunkSize, and scanRowBudget to return results in smaller segments instead of loading the full result set at once.

7. Results and Analysis

Optimisation

7.1 Client-side query timing benchmark

This test measured client-side fetch time for different query filters under the same workload. Each query case was executed 5 times. In every run, the client submitted the query to Node A and performed 40 FetchChunk calls with max_rows = 10. This fetched up to 400 rows per run. The table reports two values: **Avg Total Fetch Time**, which is the average time to complete all 40 fetch calls, and **Avg Latency per FetchChunk Call**, which is the average time for one fetch request.The fetch count was fixed so that Q1–Q5 used the same workload.

Table 1: Client-Side Query Timing Under Fixed Fetch Workload

Query Case	Filter	Avg total Fetch Time (ms)	Avg Latency per FetchChunk call (ms)	Sources observed
Q1	No filter	1855.41	46.39	A-I
Q2	Payment type =1	1883.00	47.08	A-I
Q3	Distance 0-1 mile	1901.28	47.53	A-I
Q4	Total $20-$50	1911.31	47.78	A-I
Q5	Combined filter	1910.43	47.76	A-I

The average latency per FetchChunk call stayed between **46 ms and 48 ms** for all query cases. This shows that segmented fetch performance was stable across different filters. The average total fetch time was around **1.8–1.9 seconds** because each run performed 40 sequential fetch calls.

Since every query case used the same workload of **40 FetchChunk calls with max_rows = 10**, the results compare fixed-size segmented fetch latency rather than full dataset completion time. The **Sources Observed** column shows **A–I** for every query case, confirming that all nine shards participated while Node A remained the only client-facing node.

7.2 Client-side segmented response behavior

This test measured how the number of rows returned per FetchChunk request to see how chunk size affects performance. The client ran the same broad query with different max_rows values and kept the target output close to 400 rows. Here, Avg Total Fetch Time means the average time to complete all fetch calls for that run, and Avg Latency per FetchChunk Call means the average time for one individual fetch request.

Table 2: Client-Side Segmented Response Behavior with Different Chunk Sizes

Max rows per Fetch	Avg Fetch Calls	Avg Total Fetch Time (ms)	Avg Latency per FetchChunk (ms)	Source Nodes Observed
10	40	1865.88	46.65	A–I
50	8	402.04	50.25	A, B, C, G, H, I
100	4	206.10	51.53	A, B, G, H

The results show the tradeoff between **chunk size**, **round-trip overhead**, and **overlay source coverage**. When max_rows is small, each FetchChunk returns fewer rows, so the client must issue more repeated fetch requests to reach the target sample size. This increases total fetch time because each request adds RPC, coordination, and response-handling overhead.

When max_rows is larger, each FetchChunk can return more rows, so the client reaches the same target sample with fewer client-server round trips. This reduces total fetch time. However, source coverage may be lower because the fixed-size sample can complete before all overlay branches contribute results. Smaller chunks expose distributed participation more clearly, while larger chunks improve fetch efficiency.

7.3 Client-side overlay coverage

This test verified whether recursive overlay fetching reached all distributed nodes. The client submitted a broad query only to Node A and used max_rows = 10 while fetching enough chunks to record all unique source nodes from the returned row metadata.
Although the client connected only to Node A, the returned rows included sources from all nodes A–I. This confirms that Node A was not serving only its local shard; it was collecting results through recursive overlay forwarding. It also verified deeper overlay paths, such as: F -> E -> B -> A -> Client.

7.4 Server-side processing time

This test measured internal processing time inside Node A's RPC handlers while running the same fixed client benchmark. Each query case was executed 5 times. In each run, the client submitted the query to Node A and then performed 40 repeated FetchChunk calls with max_rows = 10. Therefore, these results measure server-side processing for a fixed segmented fetch workload, not full dataset completion time.

Average SubmitQuery Time measures the time spent in the SubmitQuery handler, where Node A initializes request state and returns a request_id. Average Server Fetch Time measures the total server-side time spent across all FetchChunkcalls for one request, averaged across the 5 runs.

7.4 Server-side processing time

This test measured internal processing time inside Node A's RPC handlers while running the same fixed client benchmark. Each query case was executed 5 times. In each run, the client submitted the query to Node A and then performed 40 repeated FetchChunk calls with max_rows = 10. Therefore, these results measure server-side processing for a fixed segmented fetch workload. Mean Submit Time measures the time spent in the SubmitQuery handler, while Mean Server Fetch-All Time is the total server-side time spent across all FetchChunk calls for a request, averaged across the 5 runs.

Table 3: Server-Side Query Processing Time in Node A

Query Case	Filter	Average SubmitQuery Time	Average ServerFetchAll Time For Fixed fetches (ms)

Q1	No filter	0.060	21.029
Q2	Payment type =1	0.025	23.991
Q3	Distance 0 -1 mile	0.023	40.334
Q4	Total $20-$50	0.023	33.159
Q5	Combined filter	0.032	39.559

SubmitQuery time stayed close to zero for all query cases because Node A only created the request state and returned the request identifier. The main work happened during FetchChunk, where Node A served result segments and coordinated with the overlay when needed.

The fetch time increased for more selective filters because the server had to scan more rows before finding enough matching records to fill the requested chunks. This is why Q3, Q4, and Q5 show higher server fetch time than Q1 and Q2. Overall, the results show that server-side cost is dominated by chunk fetching and scan effort, while query submission adds very little overhead.

Baseline

7.1 Client-side query timing benchmark

Table 4: Baseline Client-Side Query Timing Under Fixed Fetch Workload

Query Case	**Filter**	**Avg total Fetch Time (ms)**	**Avg Latency per FetchChunk call (ms)**	**Sources observed**
Q1	No filter	3462.03	86.55	A-I
Q2	Payment type =1	3450.2	86.25	A-I
Q3	Distance 0-1 mile	3460.41	86.51	A-I
Q4	Total $20-$50	3456.9	86.42	A-I
Q5	Combined filter	3453.47	86.34	A-I

The average fetch latency remained stable at approximately 86ms per fetch across all query cases. Since all queries traversed the full overlay and returned results from nodes A-I, the dominant cost was the overhead of repeated RPC calls rather than filter complexity.

Compared to the optimized version, the baseline exhibits higher per-fetch latency, as each fetch is part of a heavier end-to-end process involving cached result access and RPC overhead.

7.2 Client-side segmented response behavior

Table 5: Baseline Client-Side Segmented Response Behavior with Different Chunk Sizes

Max rows per Fetch	Avg Fetch Calls	Avg Total Fetch Time (ms)	Avg Latency per FetchChunk (ms)	Source Nodes Observed
10	16	4.74092	0.277354	A–I
50	4	2.59754	0.619979	A–I
100	2	2.21796	1.06179	A–I

The results show the expected tradeoff between chunk size and fetch frequency with smaller chunks requiring more fetch calls but a lower latency per call and larger chunks needing fewer calls but exhibit higher latency per call.

Unlike the optimized system, the baseline performs minimal work during fetch because results are already precomputed. As a result, total fetch times remain very small, indicating that fetch operations primarily involve reading cached results rather than executing distributed computation.

7.3 Client-side overlay coverage

For all query cases, the returned rows included sources from nodes A-I confirming that all shards participated in query execution and that Node A correctly aggregated results from all nodes. Proper recursive forwarding and aggregation were demonstrated.

7.4 Server-side processing time

Table 6: Baseline Server-Side Query Processing Time in Node A

Query Case	Filter	Average SubmitQuery Time	Average ServerFetchAll Time For Fixed fetches (ms)
Q1	No filter	14.709	3.292
Q2	Payment type =1	14.876	3.242
Q3	Distance 0 -1 mile	20.99	3.24
Q4	Total $20-$50	19.415	3.196

Q5	Combined filter	17.878	3.24

In the baseline system, SubmitQuery performs the majority of computation while FetchChunk has minimal processing overhead. Submit times are significantly higher than fetch times because the baseline executes the entire distributed query during submission. By the time fetch operations occur, all results have already been computed and stored in memory.
Fetch-all times remain consistently low (~3ms), confirming that fetch operations primarily involve reading from cached results rather than performing additional computations.

8. Comparative Analysis of Baseline and Optimized Design

The baseline and optimized systems use the same 9-node overlay but follow different execution strategies. The baseline is precompute-oriented — most query work happens during SubmitQuery, and FetchChunk calls mostly return already-prepared results. The optimized system is segmented — SubmitQuery is lightweight, and processing happens incrementally through repeated FetchChunk calls.

This is visible in the benchmark numbers. In a fixed client benchmark of 40 FetchChunk calls with max_rows = 10, the optimized version averaged **46–48 ms per fetch** vs **86–89 ms** in the baseline, resulting in a roughly **46% improvement**. Each fetch call in the optimized system is lighter because it processes data in bounded chunks rather than reading from a large precomputed buffer.

The baseline's higher per-fetch time comes from serving precomputed results. It works fine for smaller result sets where upfront computation doesn't create memory pressure — results are ready before fetching starts, so the client doesn't wait.

However, the primary difference is how work is structured and synchronized across the distributed system. In the baseline, SubmitQuery introduces a global synchronization barrier: Node A must wait for all nodes in the overlay to complete their full shard scans before returning any results. This rates a long critical path determined by the slowest node, and requires aggregating and storing the full result set in memory.

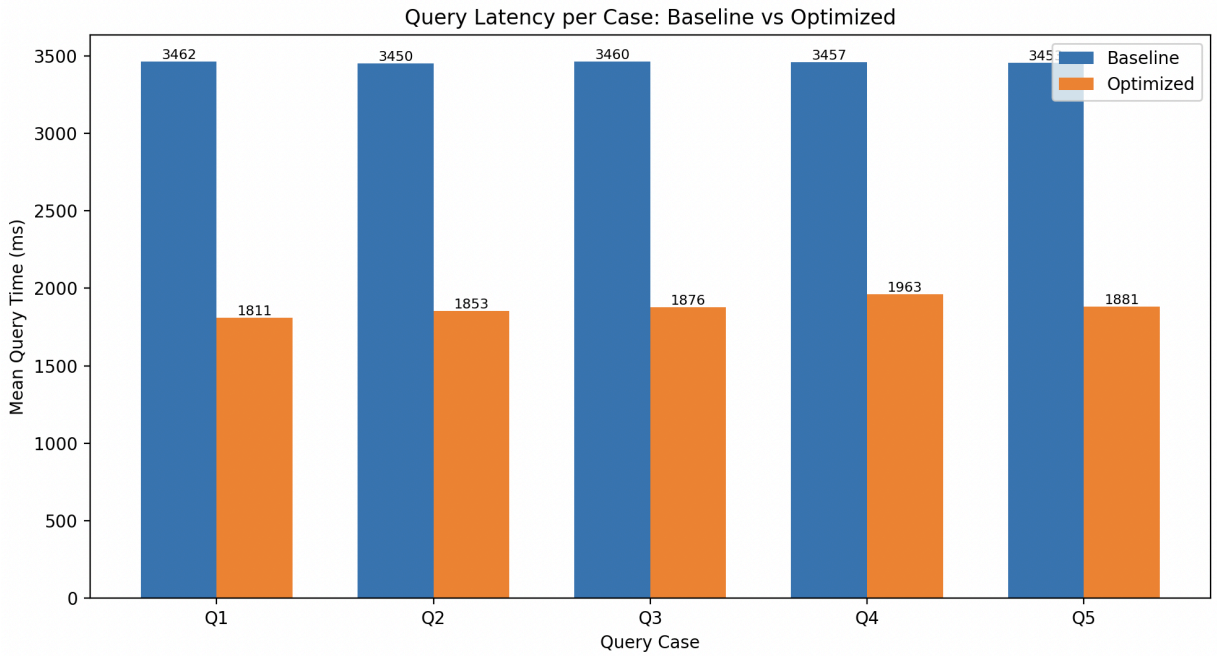
In contrast, the optimized design removes the synchronization barrier by performing bounded, demand-driven computation during FetchChunk. Each fetch only requires enough data to fill a chunk, allowing nodes to terminate scanning early instead of processing entire shards. This reduces the effective work per request and shortens the critical path.

The optimized design uses startRow, chunkSize, and scanRowBudget to bound each fetch. This avoids materializing the full result set at once and supports incremental delivery. This shift also reduces data movement and memory pressure. In the baseline, all matching rows are transmitted and stored at Node A, increasing serialization overhead and memory usage. In the optimized system, only chunk-sized data is transferred per request, improving cache locality and reducing communication overhead. Additionally,

computation and communication can overlap, as subsequent fetches continue processing while previous results are consumed.

Weighted shard assignment distributes partitions by data weight rather than count — and this is where it actually earns its benefit. Natural file-based sharding creates real imbalance; monthly taxi CSV files differ in trip count and file size, so round-robin assignment would overload some workers. Weighted assignment accounts for this, which shows up as more consistent latency across Q1–Q5. By accounting for shard size and trip count, the optimized system prevents large shards from concentrating on a single node, which reduces variability in execution time across the overlay. This contributes to the stable fetch latency observed across different cases, even when filters vary.

Baseline is simpler and faster when precomputed results are small. The optimized design handles larger or uneven workloads better, where load balance, bounded memory, and incremental progress matter. Overall, the improvement comes from restructuring execution from coarse-grained, fully synchronized processing to fine-grained, incremental processing, resulting in a reduced coordination overhead and improved scalability.



9. Conclusion

This project explored the design and evaluation of a distributed query processing system built on a multi-node overlay architecture. Starting from a baseline implementation, the system was extended with several optimizations to address key limitations related to performance, memory usage, and workload distribution.

The baseline system demonstrated a functional distributed query pipeline, where all computation was performed during a SubmitQuery and results were fully established before any client fetch operations.

This design provided fast fetch responses due to precomputed results but resulted in high submission latency and increased memory usage, especially for larger datasets.

A segmented execution model was introduced in the optimized version, shifting computation from submission time to on-demand processing during FetchChunk. Visited-node tracking, weighted shard assignment, adaptive chunking, and load-aware scheduling techniques were incorporated, improving the system's overall efficiency. The results showed that the optimized version reduced per-fetch latency and provided more consistent performance across different query cases, while also lowering memory pressure through bounded, incremental result delivery.

Experimental results confirmed the trade-offs between the two approaches. The baseline system performs well for smaller workloads where precomputation overhead is manageable, but it does not scale effectively due to its eager execution model. In contrast, the optimized system achieves better scalability by distributing work over time and across nodes, enabling fairer resource utilization and more efficient handling of large or imbalanced datasets. These findings demonstrate that careful coordination of data placement, workload distribution, and query execution can lead to meaningful performance improvements in distributed data environments.